

Python para Economistas

Universidad Nacional de La Plata
Maestría de Economía

Belén Michel Torino
Rodrigo Bonazzola

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Dato Numéricos

Enteros: o *integer* son valores numéricos enteros, positivos o negativos, de longitud ilimitada.

```
1 4
2 46821973
3 -785
```

De coma flotante: o *float* son valores numéricos con decimales, positivos o negativos. En su versión nativa guarda hasta 16 lugares decimales.

```
1 4.0214
2 4682.0008783
3 -785.1
```

Dato de Texto

Caracteres: son valores enmarcados entre ' ' o . Al estar enmarcados, los valores son interpretados como texto (y no como números u operadores aritméticos).

```
1 "A"  
2 "t"  
3 'r'  
4 "1"
```

Cadenas de Caracteres: o *string* son una sucesión de caracteres enmarcados entre ' ' o .

```
1 "Hola!"  
2 "Nos, los representantes del pueblo de la"
```

Booleano

Lógico: o *booleano* son expresiones que se evalúan a ser ciertas o no, es decir, adoptan uno de dos valores: True o False.

```
1 >>> "texto" == "texto"
2 True
3 >>> "texto" != "texto"
4 False
5 >>> 256 < 76
6 False
7 >>> 256 >= 76
8 True
9 >>> 256 >= 256
10 True
```

Tipos de datos: preguntas

Ahora veamos algunas preguntas para asentar estas definiciones. Abran una consola o un Jupyter Notebook y prueben lo siguiente para poder responder las preguntas.

¿Para Python, es igual un número escrito como entero o en formato de coma flotante? ¿Y si está escrito cómo string? Prueben lo siguiente:

```
1 >>> 17 == 17.0
2
3 >>> 524 == "524"
```

Tipos de datos: preguntas II

¿Los strings, o cadena de caracteres, son sensibles al uso de o ' '?
Prueben lo siguiente:

```
1 >>> "Argentina" == 'Argentina'
```

¿Los strings, o cadena de caracteres, son sensibles al uso de mayúsculas y minúsculas? Prueben lo siguiente:

```
1 >>> "Argentina" == "ARGENTINA"
```


Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Operaciones básicas

Ahora veamos algunas operaciones básicas!

Sumar y Restar:

```
1 >>> 100 + 45
2 145
3 >>> 100.5 - 12.3
4 88.2
5 >>> suma_mins = 15 + 60
6 >>> suma_mins += 60 #Equivale a suma_mins = suma_mins + 60
7 ??
```

Multiplicar y Dividir:

```
1 >>> 10.1 * 3
2 30.3
3 >>> 80 / 4
4 20.0
```

Nota: división entre enteros da como resultado un float en Python 3.x.

En versiones anteriores resultaba en un entero.

Operaciones básicas II

Exponencial:

```
1 >>> 12**2
2 144
```

Cociente de una división:

```
1 >>> 10.0 // 4
2 2
```

Módulo (resto de una división):

```
1 >>> 5 % 2
2 1
3 >>> 10 % 2
4 0
5 >>> 4.5 % 4
6 0.5
```

Nota: en breve veremos un caso dónde esto es útil.

Operaciones con strings

Con *strings* también podemos hacer algunas operaciones

Concatenar (sumar):

```
1 >>> "Esto es " + "concatenar dos strings"  
2 "Esto es concatenar dos strings"  
3 >>> "a" + "bb" + "ccc"  
4 "abbccc"
```

Repetir (multiplicar):

```
1 >>> 'Muy bueno!'*2  
2 'Muy bueno!Muy bueno!'
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos**
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Operadores lógicos

Operadores lógicos: son *and*, *or* y *not* y dan como resultados un valor booleano (True o False).

Van a ser particularmente útiles para las sentencias condicionales.

```
1 >>> 4 < 7 and 4 > 1
2 True
3 >>> 4 == 1 or 4 > 1
4 True
5 >>> 4 == 1 or 4 > 9
6 False
7 >>> not (4 == 1)
8 True
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Variables

Variable: Son objetos con un *identificador* (nombre) y un contenido asociados. En Python son fáciles de definir, se hace así:

```
1 >>> mivariable = "a"
2 >>> print(mivariable)
3 "a"
4 >>> a = 100
5 >>> b = 17
6 >>> print(a + b)
7 117
```

Es una buena práctica que los nombres de las variables tengan un sentido asociado a su contenido.

```
1 >>> nombre_universidad = "Universidad de San Andres"
2 >>> print(nombre_universidad)
3 "Universidad de San Andres"
```


Variables: restricciones

Hay algunas consideraciones a tener en cuenta respecto a los nombres que se pueden utilizar al definir una variable:

- Es sensible al uso de mayúsculas/minúsculas (no son intercambiables).
- Solo puede contener valores alfanúmericos o " _"
- No puede comenzar con un caracter numérico
- No usar palabras reservadas (ej. True o False)

1

```
>>> help("keywords")
```

Algunas buenas prácticas son:

- siempre minusculas a menos que sea una constante (global en stata)
- se usa snake_case y no CamelCase
- no usar caracteres latinos (ej: "ó", "ñ", "ü", etc.).
- explicativa del valor que guarda

Operaciones y variables: preguntas

Vimos que el operador `-` se puede usar para valores numéricos y para strings. ¿Se puede usar para una combinación de ambos?

```
1 >>> 12 + "34"
```

Al nombrar dos variables de manera similar pero modificando mayúsculas/minúsculas, ¿estamos creando dos variables o pisando el valor de la primera con el valor de la segunda? *Prueben creando ambas variables y luego imprimiendo sus valores.*

```
1 >>> resultado = "a"
```

```
2 >>> RESULTADO = "b"
```

¿Alguno de estos nombres de variable es válido?

```
1 >>> 2variable = "a"
```

```
2 >>> mi-variable = "a"
```

```
3 >>> mi__var_iab_ = "a"
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

¿Que son las colecciones?

Una colección en python es un container que permite agrupar objetos y darles un nombre.

Ej. Lista de código de paises:

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
```

Hoy vamos a ver los cuatro tipos de collecciones nativas de python: listas, diccionarios, sets y tuplas.

¿Porqué tantos tipos de colecciones?

- Ordenadas vs. sin orden
- Valores repetidos vs. valores unicos
- Mutables vs. inmutables

Es una secuencia **ordenada**, **mutable** que admite **valores repetidos** y **distintos tipos** de elementos

- ¿Cómo definir una lista? con []

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']  
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045.5, 3462]
```

- ¿saber el largo de una lista? len()

```
1 >>> len(paises)  
2 6
```

- ¿Cómo acceder a un elemento? indexando (desde 0 a N o -1).

```
1 >>> paises[0]  
2 'ARG'  
3 >>> pob_m[-1]  
4 3462
```

- ¿Cómo acceder a un sub-conjunto de elementos?

```
1 >>> paises[:]  
2 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']  
3 >>> paises[4:]  
4 ['PRY', 'URY']  
5 >>> paises[:3] #hasta la pocision 3 sin incluir  
6 ['ARG', 'BOL', 'BRA']  
7 >>> paises[2:-2] #equivalente a paises[2:4]  
8 ['BRA', 'CHL']
```

- ¿Cómo modificar un elemento?

```
1 >>> pob_m[-2] = 7045  
2 >>> print(pob_m)  
3 [44939, 11513, 211050, 18952, 7045, 3462]
```

Listas: métodos básicos

- Agregar objetos: append, insert.

```
1 >>> paises.append('PER')
2 >>> pob_m.insert(6, 31989)
3 >>> print("La poblacion de ", paises[-1], pob_m[-1])
4 La poblacion de PER 31989
```

- Unir listas: extend

```
1 >>> paises2 = ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
2 >>> paises.extend(paises2)
3 >>> print(paises)
4 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY', 'PER', 'COL',
   'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
```

- Eliminar objetos: remove, del, pop.

```
1 >>> paises.remove('COL')
2 >>> paises.pop(-1)
3 'VEN'
4 >>> del paises[-4:]
```

Listas: copias y referencias

- ¿Cómo hacer una referencia de una lista?

```
1 >>> paises2 = ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
2 >>> paises_sa = paises2
3 >>> paises_sa.append('ARG')
4 >>> print(paises_sa)
5 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG']
6 >>> print(paises2)
7 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG']
```

- Sí no era esto lo que queríamos, saquemos ARG y hagamos una copia independiente de la lista paises2:

```
1 >>> paises2.remove('ARG')
```


- ¿Cómo copiar una lista?

```
1 >>> paises_sa = paises2.copy() # equivalente a:  
   paises_sa = paises2[:]  
2 >>> paises_sa.extend(paises)  
3 >>> print(paises2)  
4 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'PER']  
5 >>> paises_sa  
6 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG', 'BOL',  
   'BRA', 'CHL', 'PRY', 'URY', 'PER']
```

- ¿Cuál es el resultado de sumar la lista de países con la lista de población?

```
1 >>> paises + pob_m
2 ???
```

```
1 >>> paises.append(pob_m)
2
3 >>> paises.extend(pob_m)
```

Tuplas

Es una secuencia **ordenada**, **inmutable** que admite **valores repetidos** y **distintos tipos** de elementos. Se utilizan para mantener juntos varios objetos.

- ¿Cómo definir una tupla? con ()

```
1 >>> argentina = ('ARG', 44939)
2 >>> bolivia = ('BOL', 11513)
3 >>> brasil = ('BRA', 211050)
```

- ¿Cómo modificar, agregar o eliminar un elemento? No se puede.

```
1 >>> argentina[0] = 'ARGEN'
2 Traceback (most recent call last):
3   File "<stdin>", line 1, in <module>
4 TypeError: 'tuple' object does not support item
  assignment
```

```
1 >>> argentina = ('ARG', 44939, 22947)
2 >>> bolivia = ('BOL', 11513, 9086)
3 >>> brasil = ('BRA', 211050, 15259)
```

Tuplas: índices

- ¿Cómo saber el largo de una tupla? `len( )`

```
1 >>> len(brasil)
2 3
```

- ¿Cómo acceder a un elemento? indexando (desde 0 a N o -1).

```
1 >>> bolivia[0]
2 'BOL'
3 >>> bolivia[1] #equivalente a bolivia[-2]
4 11513
```

- ¿Cómo acceder a un sub-conjunto de elementos? Igual a listas

```
1 >>> argentina[:]
2 ('ARG', 44939, 22947)
3 >>> argentina[1:]
4 (44939, 22947)
5 >>> argentina[:2] #equivalente a argentina[:-1]
6 ('ARG', 44939)
```

Conjuntos (sets)

Es una secuencia **no ordenada**, **mutable** de **distintos tipos** de elementos **no repetidos**.

- ¿Cómo definir un set? con `set([ ])`

```
1 >>> chl_limit = set(['ARG', 'BOL', 'PER'])
2 >>> arg_limit = set(paises)
3 >>> print(arg_limit)
4 {'URY', 'BOL', 'ARG', 'CHL', 'PRY', 'BRA'}
5 >>> type(arg_limit)
6 <class 'set'>
```

- ¿Cómo saber el largo de un set? `len( )`

```
1 >>> len(chl_limit)
2 3
```

Sets: no ordenados

- No se puede acceder ni modificar un elemento

```
1 >>> chl_limit[:]  
2 Traceback (most recent call last):  
3   File "<stdin>", line 1, in <module>  
4 TypeError: 'set' object is not subscriptable
```

```
1 >>> chl_limit[-2] = 'AAA'  
2 Traceback (most recent call last):  
3   File "<stdin>", line 1, in <module>  
4 TypeError: 'set' object does not support item assignment
```

Sets: métodos básicos

- Agregar objetos: `add()`

```
1 >>> arg_limit.add('ARG')
2 >>> chl_limit.add('CHL')
```

- Unir dos sets: `union()`

```
1 >>> ar_cl_limit = arg_limit.union(chl_limit)
2 >>> print(ar_cl_limit)
3 {'URY', 'BOL', 'ARG', 'CHL', 'PER', 'PRY', 'BRA'}
```

- Eliminar objetos: `remove`, `del`, `pop`.

```
1 >>> arg_limit.remove('ARG')
2 >>> chl_limit.discard('CHL')
3 >>> arg_limit.pop()
4 'URY'
5 #volvamos a agregarlo: >>> arg_limit.add('URY')
```

- ¿Cómo saber la intersección y las diferencias de dos sets?

```
1 >>> arg_limit.difference(chl_limit)
2 {'CHL', 'URY', 'BRA', 'PRY'}
3 >>> arg_limit.intersection(chl_limit)
4 {'BOL'}
```

- ¿Cómo saber si un set es subset de otro o disjunto?

```
1 >>> chl_limit.issubset(arg_limit)
2 False
3 >>> chl_limit.isdisjoint(arg_limit)
4 False
```


Sets: copias y referencias

- ¿Cómo hacer una referencia de un set?

```
1 >>> pry_limit = arg_limit
2 >>> pry_limit.remove('CHL')
3 >>> arg_limit
4 {'URY', 'BOL', 'PRY', 'BRA'}
```

```
1 >>> arg_limit.add('CHL') #primero agreguemos Chile
```

- ¿Cómo hacer una copia independiente del set?

```
1 >>> pry_limit = arg_limit.copy()
2 >>> pry_limit.remove('CHL')
3 >>> arg_limit
4 {'URY', 'BOL', 'CHL', 'PRY', 'BRA'}
```

Sets y Tuplas: pregunta

- ¿El valor de temp1 y temp2 será el mismo?

```
1 >>> pry_limit
2 {'URY', 'BOL', 'PRY', 'BRA'}
3 >>> temp1 = pry_limit.remove('URY')
4 >>> temp2 = pry_limit.pop()
5
```

- ¿Cual es un posible valor de temp2?

```
1 a) {'URY', 'BOL', 'PRY'}
2 b) {'BOL', 'PRY'}
3 c) 'URY'
4 d) 'BRA'
5
```

Los diccionarios son conjuntos de **claves:valores** (key:values). Las claves deben ser **únicas y hashables**. Los **valores se pueden modificar** pero las claves no.

- ¿Cómo definir un diccionario? con { }

```
1 >>> arg_carac = {'cod': 'ARG', 'pob': 44939, 'gdp': 'N'}
2 >>> bol_carac = {'cod': 'BOL', 'pob': 11513, 'gdp': 'N'}
3 >>> print(bol_carac)
4 {'cod': 'BOL', 'pob': 11513, 'gdp': 'N'}
```

- ¿saber el largo de un diccionario? len()

```
1 >>> len(bol_carac) #equivalente a len(bol_carac.items())
2 3
```

- ¿Cómo acceder a un elemento? por la clave

```
1 >>> bol_carac['cod']  
2 'BOL'  
3 >>> arg_carac['pob']  
4 44939
```

- ¿Cómo cambiar un valor?

```
1 >>> arg_carac['gdp'] = 1031215  
2 >>> bol_carac['gdp'] = 104609
```

- ¿Cómo crear una nueva clave:valor?

```
1 >>> arg_carac['gdp_pc'] = arg_carac['gdp']/  
2                             arg_carac['pob']*100
```

- ¿Cómo borrar una clave:valor?

```
1 >>> dic_pais.pop("gdp_pc")
```

- ¿Cómo saber las claves o valores de un diccionario?

```
1 >>> bol_carac.keys()
2 dict_keys(['cod', 'poblacion', 'gdp'])
3
4 >>> bol_carac.values()
5 dict_values(['BOL', 11513, 104609])
6
7 >>> bol_carac.items()
8 dict_items([('cod', 'BOL'),
9             ('poblacion', 11513),
10            ('gdp', 104609)])
```

item ¿Cómo saber si una clave esta en un diccionario?

```
1 >>> 'cod' in bol_carac.keys()
2 True
```

- ¿Cómo obtener un valor cuando no se si la clave existe?

```
1 >>> arg_carac["gdp_pc"]
2 22946.99481519393
3
4 >>> bol_carac["gdp_pc"]
5 Traceback (most recent call last):
6   File "<stdin>", line 1, in <module>
7 KeyError: 'gdp_pc'
```

```
1 >>> bol_carac.get("gdp_pc", 'No value')
2 'No value'
```

- ¿Cómo crear un diccionario con los datos de una lista?

```
1 >>> print(países)
2 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
3 >>> info_países = dict.fromkeys(países, "")
4 >>> info_países
5 {'ARG': '', 'BOL': '', 'BRA': '', 'CHL': '', 'PRY': '',
  'URY': ''}
```

```
1 >>> print(pob_m)
2 [44939, 11513, 211050, 18952, 7045, 3462]
3 >>> poblacion = dict(zip(países, pob_m))
4 >>> poblacion
5 {'ARG': 44939, 'BOL': 11513, 'BRA': 211050, 'CHL':
  18952, 'PRY': 7045, 'URY': 3462}
```

- Un diccionario puede ser valor de otro diccionario

```
1 >>> bra_carac = {'cod': 'BRA', 'pob': 211050, 'gdp': 3220373}
2 >>> chl_carac = {'cod': 'CHL', 'pob': 18952, 'gdp': 476738}
3 >>> pry_carac = {'cod': 'PRY', 'pob': 7045, 'gdp': 93062}
4 >>> ury_carac = {'cod': 'URY', 'pob': 3462, 'gdp': 77732}
5
6 >>> paises_info = { 'argentina': arg_carac,
7                     'bolivia': bol_carac,
8                     'brasil': bra_carac,
9                     'chile': chl_carac,
10                    'paraguay': pry_carac,
11                    'uruguay': ury_carac}
12
13 >>> paises_info['brasil']['gdp']
14 3220373
```


Cuadro 1: Resumen de colecciones

	Listas	Tuplas	Sets	Diccionarios
Se definen con	[]	()	set()	{ }
Ordenados	Si	Si	No	No
Inmutables	No	Si	No	keys: Si, values: No
Elementos unicos	No	No	Si	keys: Si, values: No
Usos	Para colecciones simples, iterables que se modifican seguido	Para datos que no van a cambiar. Puede ser la key de un diccionario.	Para chequear pertenencia y eliminar duplicados. Para elementos unicos.	Para asociaciones lógicas de key:value Para busquedas rápidas

- ¿Que pasa si intento crear un set de un diccionario?

```
1 >>> set_arg_carac = set(arg_carac)
2 >>> print(set_arg_carac)
3
4 a) error
5 b) {'cod', 'pob', 'gdp'}
6 c) {'cod', 'ARG', 'pob', 44939, 'gdp', 1031215}
7 d) {'arg_carac'}
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Bucles: for loop

Bucle: o *loop* es cuando usamos una sentencia para indicarle al intérprete que repita una serie de instrucciones hasta que se deje de cumplir una regla definida.

for loop: para cada X de Y, hacer Z.

```
1 >>> # armo una lista de paises
2 >>> lista_paises = ["Argentina", "Bolivia", "Chile"]
3 >>> # construyo un for loop
4 >>> for pais in lista_paises: # el ":" es necesario
5 >>>     print("Nombre: " + pais) # la sangria es necesaria
6 Nombre: Argentina
7 Nombre: Bolivia
8 Nombre: Chile
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):  
2 >>>     print(n, ":", pais)  
3 0 : Argentina  
4 1 : Bolivia  
5 2 : Chile
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):  
2 >>>     print(n, ":", pais)  
3 0 : Argentina  
4 1 : Bolivia  
5 2 : Chile
```

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']  
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045, 3462]
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):
2 >>>     print(n, ":", pais)
3 0 : Argentina
4 1 : Bolivia
5 2 : Chile
```

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045, 3462]
```

```
1 >>> for n, pais in enumerate(paises):
2 >>>     print(pais, ': ', pob_m[n])
3 ARG: 44939
4 BOL: 11513
5 BRA: 211050
6 CHL: 18952
7 PRY: 7045
8 URY: 3462
```

Bucles: while loop

while loop: mientras X cumpla condición Y, hacer Z.

```
1 >>> # defino cupo maximo
2 >>> cupos_libres = 3
3 >>> # ahora el while loop
4 >>> while cupos_libres > 0:
5 >>>     print('Inscripcion aceptada')
6 >>>     cupos_libres -= 1
7 >>> print('Cupo lleno') # esta linea esta fuera del loop
8 Inscripcion aceptada
9 Inscripcion aceptada
10 Inscripcion aceptada
11 Cupo lleno
```


- ¿Cómo iterar sobre la lista con las poblaciones para obtener la población total de todos esos países?

Bucles: for loop

- ¿Cómo iterar sobre la lista con las poblaciones para obtener la población total de todos esos países?

```
1 >>> pob_tot = 0
2 >>> for pob in pob_m:
3 >>>     pob_tot += pob
4 >>> print(pob_tot)
5 296961
```

Control de flujo

Usamos sentencias condicionales para construir programas (scripts) que respondan a condiciones predefinidas.

sentencia condicional: if A, X, elif B, Y, else C, Z.

```
1 >>> edad = 25
2 >>> # aca empieza la sentencia condicional
3 >>> if edad < 18: # el ":" es importante
4 >>>     print("Menor de edad") # atencion a la indentaci n
5 >>> elif edad >= 18 and edad <= 30:
6 >>>     print("Mayor de edad")
7 >>> else:
8 >>>     print("No se admiten Mayores de 30")
9 Mayor de edad
```

Sentencias: juntamos lo visto

Los tipos de sentencia se pueden combinar.

```
1 >>> listado_numeros = [10,11,12,13]
2 >>> # combinamos un for loop y un if
3 >>> for numero in listado_numeros:
4 >>>     if numero % 2 == 0: # se acuerdan del modulo?
5 >>>         print(numero, "es par")
6 >>>     else:
7 >>>         print(numero, "es impar")
8 10 es par
9 11 es impar
10 12 es par
11 13 es impar
```

Dudas?